

Porting `igsca_tree` / `igsca_ctree` into `doTrees()`

What is missing, what is wrong, and what to change — branch `igscaTrees`

1 Scope and method

This reviews the port of `dev/igsca/tree_template/igsca_tree.R` and `igsca_testers.R` into `R/postestimate_doTrees.R` + `R/helper_doTrees.R`, and the new `tests/testthat/test-postestimate_doTrees.R`.

Every finding below was **executed**, not read off. All five `influence` values were run against `tests/testthat/data/igscaTrees.Rdata` (`n = 1000`, covariates `z_true` (2-level factor), `noise_1`, `noise_2`) under `devtools::load_all()`, with reduced `R_test/maxdepth/max_cuts` where the default control was too slow to finish. Verbatim error text is quoted.

Revision history. The first pass found the partition family unrunnable. Since then:

- **Second pass** — `idx_permutation()` and `ndt_dists()` were added (`R/helper_doTrees.R`:928, :959) and `devtools::document()` was run. Verified in B1 and P1.
- **Third pass** — `splitter` became a `match.arg()` string and the test file was updated to match. Verified in B2 across all 17 configurations the tests exercise, plus both guard paths.

Both blockers are now cleared and every `influence` × `splitter` combination runs. What remains is three carried-over silent failures, a test suite that still cannot detect any of them, and packaging cleanup. Section 5 sets out how to test the mixed-splitter configurations specifically; its injection recipes have been rewritten for the new string-valued API.

1.1 Status at a glance

ID	Finding	Status
B1	<code>idx_permutation()</code> / <code>ndt_dists()</code> not ported	resolved
B2	<code>splitter</code> is a function in one branch, a string in the other	resolved
S1	Broken split kernel degrades to a stump with no diagnostic	open
S2	<code>control\$bonferroni</code> silently ignored on the <code>ctree</code> path	open
S3	<code>coef()</code> / <code>plot()</code> empty for any tree that splits	open
T1	Snapshots stale (<code>trees_out</code> vs <code>trees_mx</code>)	open
T2	<code>length(tree) == 5</code> is a disguised structural assertion	open
T3	<code>expect_no_error()</code> cannot see the real failure mode	open
T4	Default control makes the partition tests unrunnable	changed
T5	Collector diagnostics and the scan cache are untested	open
P1	NAMESPACE stale	changed
P2	<code>cSEM:::</code> / <code>cSEM::</code> into the package's own namespace	open
P3	Roxygen placeholders; wrong <code>@return</code>	open
P4	Argument naming departs from package convention	open
P5	<code>fit_csem()</code> hard-codes the estimator	open
P6	No input validation	open

The test file has been updated to pass strings, so it no longer errors on the signature — but T1–T4 are untouched, and T4 is what will actually stop a run completing.

2 Resolved

2.1 B1. `idx_permutation()` and `ndt_dists()` are now available

Location: R/helper_doTrees.R:928, R/helper_doTrees.R:959

Both helpers now exist, and both satisfy the contracts their call sites require. This was the sole blocker on the partition family; it is cleared.

`idx_permutation()` — orientation is correct. The returned matrix is $R \times n$ (`nrow = R`, `ncol = n`), which is what `permutation_pvalue()` needs when it indexes `goes_left[perm[r, 1]]`. Had it come back transposed, the result would have been a recycled-length nonsense null distribution rather than an error, so this is worth having confirmed explicitly. The sample-without-replacement-within-stratum construction matches `boot::permutation.array` as the docstring claims.

`ndt_dists()` — names and argument order match the call site. It returns `c(DGc, DGi, DLc, DLi)`, and `partition_stat()` reads `unname(ds[stat_kind])` for `stat_kind` in "DGi" / "DLi". The (`Sc_pool`, `Si_pool`, `mga_fit`) signature lines up with `ndt_dists(coll$ndt_pools$Sc, coll$ndt_pools$Si, mga$fit)` at R/helper_doTrees.R:635.

Verified end to end. The partition family, previously unrunnable, now grows trees. At `R_test = 24L`, `max_cuts = 3L`, `maxdepth = 1L`, `minbucket = 200L` (14.8 s):

```
Fitted party:
[1] root
|   [2] z_true in 1: *
|   [3] z_true in 2: *

      z_true      noise_1      noise_2
statistic 0.04315312 0.001707126 0.001593087
p.value   0.04000000 0.280000000 0.083333333
criterion -0.04082199 -0.328504067 -0.087011377
```

The DLi/DGi kernels agree exactly with the reference implementations used to pre-verify the path in the first pass (`argmax` cutpoint on `noise_1 = -1.69654` for both, `-0.231765` for FIT), so the shipped `ndt_dists()` computes what that analysis assumed.

Two loose ends, both minor:

- **The `strata` argument is dead.** `idx_permutation(n, R, strata = NULL)` implements stratified permutation, but `permutation_pvalue()` (R/helper_doTrees.R:699) calls `idx_permutation(n, R)` with no `strata`. If stratified permutation within the node was intended — which would be the natural thing to want when the node is already conditioned on a covariate — it is not wired up. If it was not intended, drop the parameter rather than leave an untested branch in the package.
- **The provenance comment is now wrong.** R/helper_doTrees.R:535 still says these come from R/MGA/csem_test_helpers.R. They are defined 400 lines below it. (`dev/igsca/tree_template/csem_test_helpers.R` is now present but untracked.)

2.2 B2. `splitter` is now a single `match.arg()` string

Location: R/postestimate_doTrees.R:16--29

Implemented as recommended — one string-valued argument with an explicit "native" level, resolved once before either branch:

```
doTrees <- function(data, model, covariates,
                    influence = c("mat", "vec", "FIT", "DLi", "DGi"),
                    splitter   = c("native", "FIT", "DLi", "DGi"),
```

```

        control = igsca_tree_control() {
influence <- match.arg(influence)
splitter <- match.arg(splitter)

split_fn <- switch(splitter,
  native = NULL, FIT = split_max_fitdiff,
  DLi = split_max_dli, DGi = split_max_dgi)

```

`split_fn` is consumed by the `ctree` branch at :117 and by `grow_extree()` at :221. Both original failure modes are gone, and the structural hazard behind the second one is closed: `match.arg()` guarantees a length-1 character, so `splitter %in% c(...)` can no longer evaluate to `logical(0)` and slip through `stopifnot()` vacuously.

The retained `stopifnot()` at :167 is **not** redundant — it now does real work, rejecting "native" on the partition path, where there is no engine-native scan and a `NULL` kernel would die inside `argmax_split()`'s `tryCatch` and silently yield a stump. Both guards produce readable errors:

```

doTrees(dat, model, covs, influence = "FIT", splitter = "native")
#> Error: splitter should be any one of 'FIT', 'DLi' or 'DGi' when the influence
#>       (selector) function is one of 'FIT', 'DLi' or 'DGi'

doTrees(dat, model, covs, influence = "mat", splitter = "nope")
#> Error: 'arg' should be one of "native", "FIT", "DLi", "DGi"

```

Verified sweep. All 17 configurations the test file exercises now run, none error, and every one produces a split:

Configuration	width	n_fail (node/res)	time
mat × native	3	0 / 0	0.9 s
mat × {FIT, DLi, DGi}	3	0 / 2	1.7–2.0 s
vec × native	3	0 / 0	0.3 s
vec × {FIT, DLi, DGi}	3	0 / 0	1.5–1.8 s
{FIT, DLi, DGi} × {FIT, DLi, DGi}	2	0 / 0	14.0–15.3 s

(`ctree` rows at `R_test = 50`, `maxdepth = 2`, `max_cuts = 8`; partition rows at `R_test = 24`, `max_cuts = 3`, `maxdepth = 1`, `minbucket = 200`. The `n_fail_resample = 2` on the `mat` mixed rows is ordinary non-convergence at extreme cutpoints, not a fault.)

Two residual nits:

- **An obsolete TODO.** `R/postestimate_doTrees.R:119` still reads “Add a switch statement to make sure that the splitter can only be one of `c("FIT", "DLi", "DGi")`”. `match.arg()` now does exactly that; delete the comment.
- **influence is rebound mid-function.** :45 overwrites the character `influence` with a **function** (`influence_mat / influence_vec`), while :157 re-tests `influence %in% c("FIT", "DLi", "DGi")` on the original value. It works, because the rebinding happens inside the first branch, but the two conditions now read as though they test the same variable when one of them has been overwritten. A separate `influence_fn` local would remove the trap.

3 Silent failures

3.1 S1. A broken split kernel degrades to a stump with no diagnostic

Location: R/helper_doTrees.R:310--319

Status: still live, and now demonstrable under the current API. B1 fixed the particular kernel that was dead; it did not fix the mechanism that hid it. Injecting a throwing kernel with `local_mocked_bindings()` reproduces the exact first-pass symptom against today's code:

```
local_mocked_bindings(  
  split_max_dli = function(model, mf, subset, goes_left, ctrl) stop("kernel is  
broken"))  
tr <- doTrees(dat, model, covs, influence = "mat", splitter = "DLi", control =  
ctl_mixed)  
  
partykit::width(partykit::node_party(tr))      #> 1      (a bare stump)  
attr(tr, "igsca_info")$n_fail_resample        #> 0      (no failures recorded)  
attr(tr, "igsca_info")$n_fail_split           #> NULL   (no such counter)
```

`argmax_split()` wraps each kernel call in `tryCatch(..., error = function(e) NA_real_)`, which is correct for a genuinely non-identified candidate partition but also swallows **programming** errors. Every candidate becomes NA, `any(is.finite(stats))` is FALSE, the loop nexts through every covariate, `argmax_split()` returns NULL, and partykit reads that as “no admissible split”. A dead kernel and an unsplittable node remain indistinguishable in the returned object.

Any future kernel that throws — a `calculateDG()` signature change, a renamed argument, a typo in a new distance — fails the same silent way. It is also why the original defect had to be found by hand rather than by the suite.

Fix. Count the scans that produced nothing usable, so the two cases separate:

```
stats <- vapply(cands, function(cc) {  
  tryCatch(splitter(model, mf, subset, cc$goes_left, ctrl),  
    error = function(e) NA_real_)  
}, numeric(1))  
if (!any(is.finite(stats))) {  
  collector$n_fail_split <- collector$n_fail_split + 1L  
  next  
}
```

Add `n_fail_split` to `new_collector()` and to the `igsca_info` attribute, and warn from `doTrees()` when a non-“native” splitter was requested but every node visited recorded a failed scan. Section 5.4 gives the test that pins this down — the snippet above is exactly what makes it passable.

3.2 S2. `control$bonferroni` is silently ignored on the `ctree` path

Location: R/postestimate_doTrees.R:116 (# TODO: Return to this bonferroni problem)

Unchanged. The template had one more line after `ctree_control()`:

```
cc <- partykit::ctree_control(...)  
cc$bonferroni <- isTRUE(control$bonferroni) # <- dropped in the port
```

This is not a no-op. `ctree_control()` derives `bonferroni` from `testtype`, but the port computes `testtype` from `coin_distribution` first:

```
ctree_control()$bonferroni      TRUE   (testtype = "Bonferroni")
ctree_control(testtype = "MonteCarlo")$bonferroni FALSE
```

With the default `coin_distribution = "approximate"` the port always sends `testtype = "MonteCarlo"`, so `cc$bonferroni` is always `FALSE` regardless of what the user asked for. `igsca_tree_control(bonferroni = TRUE)` is accepted, stored, threaded through, and then discarded.

The asymmetry is the real problem: on the **partition** path `grow_extree()` copies the whole control list into `ctrl` (R/helper_doTrees.R:249--251), so `bonferroni` **does** take effect there. One argument, honoured in one branch and ignored in the other, with no diagnostic.

Fix. Restore the line. To answer the TODO on its own terms: `testtype` selects how p-values are **computed** (permutation vs asymptotic), whereas `.extree_node()` reads `ctrl$bonferroni` independently to decide whether to **multiplicity-adjust** the criterion across covariates. They are orthogonal, which is exactly why the explicit assignment is needed — otherwise choosing `coin_distribution = "approximate"` covertly forces `bonferroni = FALSE`.

3.3 S3. `coef()` and `plot()` labels are empty for any tree that splits

Location: R/helper_doTrees.R:461--477 (`coef.igsca_tree`), R/helper_doTrees.R:490--507 (`plot.igsca_tree`)

Unchanged. Both methods read `nobs / objfun` out of `info_node()` at **terminal** nodes. `partykit` only calls the trafo at nodes it attempts to split, so leaves carry no info at all:

```
r <- doTrees(dat, model, covs, influence = "mat",
             control = igsca_tree_control(R_test = 50L, maxdepth = 1L))
# node 1 -> info names: criterion,p.value,converged,objfun,object,nobs
# node 2 -> info names: NULL
# node 3 -> info names: NULL

coef(r)
#> NULL
```

`coef()` returns `NULL` for every tree with at least one split (it only appears to work on stumps, where the root **is** the leaf). `plot()` does not error — `sprintf("n = %s", NULL)` yields `character(0)` — it just draws unlabelled terminal boxes, which is worse than failing.

This is a functional gap, not just a method bug: **the returned tree contains no per-leaf IGSCA fit**, which is presumably the main thing a user wants from a SEM tree. The leaf models are never estimated.

Fix. Fit the leaves once after growing, then have both methods read from there:

```
refit_leaves <- function(tree, mf, model, indicators) {
  ids <- partykit::nodeids(tree, terminal = TRUE)
  leaf <- partykit::fitted_node(partykit::node_party(tree), mf)
  stats::setNames(lapply(ids, function(id) {
    ft <- try_fit(mf[leaf == id, indicators, drop = FALSE], model)
    list(nobs = sum(leaf == id), converged = ft$ok, object = if (ft$ok) ft$fit)
  }), ids)
}
```

Store the result in `attr(ret, "igsca_info")$leaves` and rewrite `coef.igsca_tree()` / the default `plot.igsca_tree()` FUN against it. Cost is one IGSCA fit per leaf (~ 0.6 s each here), negligible next to growing the tree.

While there: `object = ft$fit` is stored in **every inner node's** info (R/
`postestimate_doTrees.R:90, :209`). Each is a full `cSEMResults` object, so a `maxdepth = 3` tree retains
up to 7 of them. Consider dropping `object` from the stored info once the leaf fits are attached
explicitly.

4 Test-suite findings

The test file is the right shape — one file per R file, five blocks covering the five methods, matching the conventions in `MEMORY.md` — and it has been updated to the new string-valued `splitter`, so it no longer errors on the signature. These are the things that still stop it from testing anything.

4.1 T1. Snapshots are stale and will fail

`tests/testthat/_snaps/postestimate_doTrees.md` still records the printed object under the name `trees_out` (lines 4 and 24):

```
# IGSCA Trees Conditional Test on Matrix of Residuals Runs as expected
```

```
Code
trees_out
```

The tests bind `trees_mx` / `trees_vec` / `trees_NPT_FIT` / `expect_snapshot()` captures the **expression** as well as the output, so both existing blocks fail on the `Code` line alone, and the three partition blocks have no snapshot at all. Delete the file and regenerate — the signature has settled, so this is now safe to do.

4.2 T2. `length(tree) == 5` is a disguised structural assertion

Location: `tests/testthat/test-postestimate_doTrees.R:32, :78, :125, :161, :196`

```
# Dirty substitute for snapshot
expect_true(length(trees_mx) == 5)
```

`length()` on a `party` dispatches to `length.party()`, which returns the **number of nodes**, not the length of a list. Measured: the same call with `maxdepth = 1L` gives `length(r) == 3`. So this asserts “the tree has exactly 5 nodes” — a strictly weaker duplicate of the snapshot immediately below it, which will break on any legitimate change to the control defaults or the RNG stream, for reasons the comment actively misleads you about.

It also interacts badly with T4: the reduced controls needed to make the suite finish produce 2- and 3-node trees, so all five assertions fail for a reason that has nothing to do with correctness. Drop them, or replace with assertions that say what they mean:

```
expect_s3_class(trees_mx, "igsca_tree")
expect_identical(attr(trees_mx, "igsca_info")$n_fail_full, 0L)
```

4.3 T3. `expect_no_error()` cannot see the failure mode that actually occurs

Location: `tests/testthat/test-postestimate_doTrees.R:35--62` and the four analogous blocks

All five blocks still wrap two or three `doTrees()` calls in a single `expect_no_error({...})`. Two problems. First, on failure you learn that **one** of three calls errored, not which. Second and more seriously, per S1 the realistic failure is not an error at all — a broken kernel returns a stump and the block passes. That is exactly what happened in the first pass: the DLi and DGi mixed splitters were entirely non-functional and the suite reported green.

This is now **more** misleading than before, not less. Every mixed configuration currently returns `width = 3`, so the blocks pass — and they would pass identically if all three kernels were dead, as S1’s mocked-kernel demonstration shows.

Split them, and assert on the artefact rather than on the absence of an exception:

```

for (sp in c("FIT", "DLi", "DGi")) {
  tr <- doTrees(dat, model, covs, influence = "mat", splitter = sp, control =
ctl_mixed)
  expect_s3_class(tr, "igsca_tree")
  expect_gt(partykit::width(partykit::node_party(tr)), 1L) # it actually split
  expect_identical(attr(tr, "igsca_info")$n_fail_full, 0L)
}

```

`expect_gt(width, 1)` is the minimum assertion that would have caught S1. Section 5 goes further.

4.4 T4. Only the partition selectors are expensive — mixed splitters are cheap

Revised from the first pass, which over-generalised the cost. The two paths differ by an order of magnitude, because only the partition **selectors** run permutation p-values; a mixed splitter re-scans candidates but inherits COIN variable selection from libcoin. Measured on this fixture:

Configuration	Wall clock
one 2-group MGA fit	0.585 s
mixed splitter, <code>maxdepth = 2</code> , <code>max_cuts = 8</code> , <code>R_test = 50</code>	1.5–2.6 s
partition selector, <code>maxdepth = 1</code> , <code>max_cuts = 3</code> , <code>R_test = 24</code>	14.0–15.3 s
partition selector, one node at <code>max_cuts = 20</code> , <code>R_test = 500</code>	~ 15 min
partition selector, one tree at <code>maxdepth = 3</code> (default)	~ 1.8 h

All 17 calls in the test file still use a bare `igsca_tree_control()`. The eight ctree-branch calls are fine at defaults; the nine partition-branch calls are not — at ~ 1.8 h each the file is on the order of half a day. Define two controls at the top and use them by branch:

```

ctl_mixed <- igsca_tree_control(R_test = 50L, maxdepth = 2L, max_cuts = 8L)
ctl_part  <- igsca_tree_control(R_test = 49L, max_cuts = 4L, maxdepth = 2L,
                               minbucket = 100L)

```

`R_test = 49L` keeps the smallest attainable p-value at $1/50 = 0.02 < \alpha$, so splitting is still possible — worth stating explicitly in a comment, because anything below `R_test = 19L` makes $p < 0.05$ unreachable and silently guarantees a stump for reasons that have nothing to do with the data. If a full-fidelity run matters, keep it behind `skip_on_ci()`.

4.5 T5. Nothing asserts the diagnostics the port exists to collect

No test touches `n_fail_full`, `n_fail_node`, `n_fail_resample`, or `root_criteria` — the entire `new_collector()` apparatus is untested, including the node-local scan cache in `argmax_split()`, which is the one piece of genuinely subtle logic in the port (`identical()` on closures as a cache key). See section 5.5.

5 Testing the mixed-splitter configurations

The mixed pairs (COIN variable selection + a model-comparison split point) are the configurations the suite is least able to check, because their failure mode is a **plausible-looking tree**, not an exception. Three measured facts shape what a useful test can look like.

(a) **The splitfun wiring works, but it is an undocumented contract.** `doTrees()` plants its kernel in `cc$splitfun` and relies on `partykit` reading it back out of the control object. A counting wrapper confirms it fires — 9 kernel calls on a `maxdepth = 2`, `max_cuts = 8` tree (1 at the root for `z_true`, 8 at node 2 for `noise_1`) on `partykit 1.2.29`. The template’s note says “verified against `partykit 1.2.28`”, so this contract has already survived one upgrade unverified. It deserves an explicit test.

(b) **The fixture cannot falsify cutpoint choice at the root.** `z_true` is a 2-level factor, so `candidate_partitions()` returns exactly **one** candidate:

```
length(candidate_partitions(1, dat$z_true, dat$z_true, max_cuts = 20L, minbucket =
30L))
#> 1
length(candidate_partitions(2, dat$noise_1, dat$noise_1, max_cuts = 20L, minbucket =
30L))
#> 20
```

Every kernel is forced to return the same split at the root. Any root-level assertion about **which** cutpoint a mixed splitter chose is vacuous, which is why no amount of rephrasing the existing `expect_no_error()` would have worked.

(c) **Kernels do disagree, on numeric covariates.** Pinning `whichvar` to `noise_1` and running `argmax_split()` directly:

```
FIT  cutpoint on noise_1 = -0.231765
DLi  cutpoint on noise_1 = -1.69654
DGi  cutpoint on noise_1 = -1.69654
```

So a differential assertion is meaningful — but **DLi and DGi legitimately agree**, both being model-implied indicator-VCV distances. Do not assert pairwise distinctness across all three.

A note on injection. Now that B2 has made `splitter` a `match.arg()` string, a test can no longer pass a custom kernel by value. Anything that needs to observe or subvert a kernel must go through `local_mocked_bindings()`, which rebinds the symbol in the package namespace where `doTrees()`’s `switch()` resolves it. Verified working on `testthat 3.3.2` (installed): the mocked counting wrapper below records the same 9 calls the by-value version did.

5.1 Kernel unit tests — the layer that was missing

Two MGA fits each, and the failure message names the broken kernel. This is the test that would have caught the missing `ndt_dists()` in seconds:

```
node_fixture <- function() {
  ind <- parseModel(model)$indicators
  ft <- try_fit(dat[, ind, drop = FALSE], model)
  list(model = list(object = ft$fit),      # what the trafo hands the kernel
        mf    = dat,
        ctrl  = list(collector = new_collector(), model = model,
                      indicators = ind, max_cuts = 20L, minbucket = 30L))
}
```

```

test_that("every split kernel returns a finite statistic", {
  fx <- node_fixture()
  gl <- fx$mf$noise_1 <= stats::median(fx$mf$noise_1)
  for (k in c("FITdiff", "DLi", "DGi")) {
    expect_true(
      is.finite(partition_stat(k, fx$model, fx$mf, seq_len(nrow(fx$mf)), gl, fx$ctrl)),
      info = k
    )
  }
})

```

`partition_stat()` converts a missing-function error into NA + `n_fail_resample++`, so `is.finite()` catches it. Observed values at the median split are `FITdiff = -0.00398`, `DLi = 0.0439`, `DGi = 0.0136` — note the negative FIT difference, which is exactly the case `partition_select()`'s `.Machine$double.eps` floor exists to handle, and worth a regression test of its own.

5.2 `argmax_split()` returns an admissible cutpoint

```

test_that("argmax_split returns an admissible cutpoint for every kernel", {
  fx <- node_fixture(); j <- match("noise_1", names(fx$mf))
  for (k in c("FIT", "DLi", "DGi")) {
    kern <- switch(k, FIT = split_max_fitdiff, DLi = split_max_dli, DGi =
split_max_dgi)
    sp <- argmax_split(kern, fx$ctrl$collector, fx$model, fx$mf,
      seq_len(nrow(fx$mf)), j, fx$ctrl)
    expect_s3_class(sp, "partysplit")
    br <- partykit::breaks_split(sp)
    expect_true(is.finite(br), info = k)
    expect_true(br > min(fx$mf$noise_1) && br < max(fx$mf$noise_1), info = k)
  }
})

test_that("different kernels place different cutpoints", {
  fx <- node_fixture(); j <- match("noise_1", names(fx$mf))
  cut <- function(kern) partykit::breaks_split(
    argmax_split(kern, fx$ctrl$collector, fx$model, fx$mf,
      seq_len(nrow(fx$mf)), j, fx$ctrl))
  # NB: DLi and DGi legitimately agree -- both are indicator-VCV distances.
  expect_false(isTRUE(all.equal(cut(split_max_fitdiff), cut(split_max_dli))))
})

```

These call the kernels directly rather than through `doTrees()`, so they are unaffected by B2 and need no mocking. This is where cutpoint choice **must** be tested, per fact (b) — at the tree level the root gives the kernel no freedom.

5.3 Assert that `partykit` actually calls the `splitfun`

```

test_that("doTrees installs its splitfun into partykit's split search", {
  calls <- new.env(parent = emptyenv()); calls$n <- 0L
  real <- split_max_fitdiff
  local_mocked_bindings(
    split_max_fitdiff = function(model, mf, subset, goes_left, ctrl) {
      calls$n <- calls$n + 1L
      real(model, mf, subset, goes_left, ctrl)
    }
  )
  set.seed(11)

```

```
tr <- doTrees(dat, model, covs, influence = "mat",
              splitter = "FIT", control = ctl_mixed)
expect_gt(calls$n, 0L) # the contract
expect_gt(partykit::width(partykit::node_party(tr)), 1L) # it split
})
```

Verified: `calls$n == 9, width == 3`. Capturing `real <- split_max_fitdiff` **before** the mock is what avoids infinite recursion. This test is what breaks loudly if a partykit upgrade stops honouring `ctrl$splitfun`.

5.4 Negative control — give the suite teeth

Codify S1's failure mode instead of leaving it undetectable:

```
test_that("a dead split kernel is reported, not silently a stump", {
  local_mocked_bindings(
    split_max_dli = function(model, mf, subset, goes_left, ctrl) stop("kernel is
broken"))
  set.seed(11)
  tr <- doTrees(dat, model, covs, influence = "mat",
                splitter = "DLi", control = ctl_mixed)
  expect_gt(attr(tr, "igsca_info")$n_fail_split, 0L)
})
```

Confirmed to fail today, for the right reason: the mocked kernel yields `width == 1, n_fail_resample == 0`, and `n_fail_split == NULL`. It passes once S1's counter exists — which is the right relationship between a test and an open bug.

5.5 The scan cache

`argmax_split()`'s cache key is `identical()` on the splitter **closure** (R/helper_doTrees.R:294--296). A matched selector/splitter pair should reuse the selector's scan; a mismatched pair should rescan with its own kernel. Both are cheap to check at unit level with a pre-armed collector, and neither is covered today:

```
test_that("a matched pair reuses the scan and a mismatched pair rescans", {
  fx <- node_fixture(); j <- match("noise_1", names(fx$mf)); subset <-
seq_len(nrow(fx$mf))
  coll <- fx$ctrl$collector
  sc <- scan_covariate("FITdiff", j, fx$model, fx$mf, subset, fx$ctrl)
  coll$scan <- stats::setNames(list(sc), as.character(j))
  coll$scan_subset <- subset
  coll$scan_splitter <- split_max_fitdiff

  expect_identical( # matched -> cache hit
    argmax_split(split_max_fitdiff, coll, fx$model, fx$mf, subset, j, fx$ctrl),
    sc$split)
  expect_false(isTRUE(all.equal( # mismatched -> rescan
    argmax_split(split_max_dli, coll, fx$model, fx$mf, subset, j, fx$ctrl),
    sc$split)))
})
```

Note this test must **not** be combined with `local_mocked_bindings()` on the same kernels: the cache key is closure identity, and a mock would change it.

5.6 What none of this proves

The five layers above establish that the mixed splitters **run, are invoked, and discriminate**. They do not establish that a mixed splitter finds the **right** cutpoint, because this fixture has no numeric covariate with a true threshold — `z_true` is binary and `noise_*` are noise. A recovery test needs a fixture where a numeric covariate drives a known group difference at a known cutpoint, then asserts the recovered breakpoint falls within a tolerance of it. That is the only test that checks the mixed configurations are statistically correct rather than merely alive, and it is worth building if the mixed pairs are a real study arm rather than a convenience.

6 Packaging and API

6.1 P1. `document()` has been run — now prune the export list

Status: inverted. `NAMESPACE` has been regenerated and `man/` populated, so the first pass’s complaint is resolved: `igsca_tree_control` and the `split_max_*` kernels are now reachable, and `cSEM::doTrees(control = cSEM::igsca_tree_control(...))` works.

The problem is now the opposite one. 26 new symbols are exported, and most are internal machinery with no standalone meaning:

```
argmax_split csem_converged fit_csem grow_extree idx_permutation
ndt_dists ndt_pools node_group_data partition_select partition_stat
permutation_pvalue scan_covariate select_ndt_dgi select_ndt_dli
select_npt split_max_dgi split_max_dli split_max_fitdiff try_fit
candidate_partitions influence_mat influence_vec ...
```

Every one of these is now public API you are committing to keep stable, and each carries a `man/*.Rd` that R CMD check will scrutinise (see P3). Replace `@export` with `@keywords internal` or `@noRd` on all of them. B2’s string-valued `splitter` has already removed the last **user-facing** reason to expose the `split_max_*` kernels — but keep them (or their `partition_stat()` backend) reachable from the test file, since the unit tests in section 5 call them directly. A defensible public surface is `doTrees`, `igsca_tree_control`, `bdiagFit`, `root_criteria`, and the two S3 methods.

6.2 P2. `:::` and `::` into the package’s own namespace

Location: `R/helper_doTrees.R:572--573` and `R/helper_doTrees.R:608`

Unchanged:

```
Sc = cSEM:::bdiagFit(single_fit, .n_blocks = 2L, .type_vcv = "construct"),
Si = cSEM:::bdiagFit(single_fit, .n_blocks = 2L, .type_vcv = "indicator")
...
cSEM::calculateFIT(mga$fit) - cSEM::calculateFIT(model$object)
```

Both are defined in this package (`bdiagFit` at `R/helper_doTrees.R:26`, `calculateFIT` at `R/helper_assess.R:2013`). R CMD check flags the `:::` form (“a package almost never needs to use `:::` for its own objects”), and both break under `load_all()` if the installed and sourced versions diverge.

The new code makes the inconsistency starker rather than better: `ndt_dists()` (`R/helper_doTrees.R:960--966`) calls `bdiagFit()`, `calculateDG()` and `calculateDL()` bare and correctly, sitting 390 lines below `ndt_pools()` doing the same thing through `cSEM:::`. Drop the prefixes.

6.3 P3. Roxygen placeholders — now materialised as `.Rd` files

`document()` has turned the placeholder blocks into committed documentation. `man/idx_permutation.Rd` in full:

```
\title{Title}
\usage{idx_permutation(n, R, strata = NULL)}
\arguments{
  \item{strata}{}
}
\description{
  Permutation resampling: sample WITHOUT replacement, within each stratum, ...
}
```

Three R CMD check problems in nine lines: the title is literally `Title`, `n` and `R` are undocumented while `strata` has an empty description, and there is no `\value{}`. That pattern repeats across the 20-odd new `.Rd` files. P1's pruning is the cheapest fix — `@noRd` deletes the file and the warnings with it — and whatever survives as public API needs a real title, real `@param` text and a real `@return`.

Two specific errors in `man/doTrees.Rd`:

- **\value is wrong.** It claims class `modelparty` and `party`. Actual class is `c("igsca_tree", "constparty", "party")` on both paths; there is no `modelparty` anywhere in the package.
- **Half the arguments are undocumented.** `data`, `model` and `covariates` did inherit from `csem_arguments` (roxygen matched them to the dot-prefixed `.data` / `.model` / `.covariates` entries). `influence`, `splitter` and `control` did not — they appear nowhere in `\arguments{}`, which is an R CMD check WARNING. They are also the three arguments a reader most needs explained, since `influence` selects between two entirely different algorithms and `splitter` is constrained by which one you picked (B2's `stopifnot`).

6.4 P4. Argument naming departs from the package convention

Every other exported cSEM entry point takes dot-prefixed arguments

(`.data`, `.model`, `.approach_weights`, ...). `doTrees()` takes bare `data`, `model`, `covariates`, `influence`, `splitter`, `control`.

(The first pass claimed this also broke `@inheritParams`; that was wrong — roxygen matched the dotted entries to the undotted parameters, as P3 shows. The case for renaming rests on API consistency alone, which is still a good enough reason while the branch is young. Doing it together with B2's now-settled signature would keep the churn to one commit.)

6.5 P5. `fit_csem()` hard-codes the estimator with no override

Location: `R/helper_doTrees.R:152--164`

```
fit_csem <- function(.data, .model, .id = NULL) {  
  csem(.data = .data, .model = .model, .id = .id,  
    .approach_weights = "GSCA", .disattenuate = TRUE,  
    .conv_criterion = "sum_diff_absolute", .iter_max = 100,  
    .GSCA_modes = "CCMP", .tolerance = 0.0001)  
}
```

Reasonable as a study default, but `doTrees()` gives the user no way past it — no ..., no argument. Since the tree machinery is estimator-agnostic (nothing outside `partition_stat()`'s FIT branch is GSCA-specific), threading ... from `doTrees()` through `try_fit()` into `csem()` costs almost nothing and makes `doTrees()` usable for PLS-PM trees too. At minimum, document that the estimator is fixed — especially now that `fit_csem` is exported.

6.6 P6. No input validation

`doTrees()` validates `influence` and `splitter` (via `match.arg`) and nothing else. Worth adding, since all three cases currently produce errors from deep inside `partykit` or `csem()`:

```
stopifnot(  
  "`data` must be a data.frame" = is.data.frame(data),  
  "`covariates` must be columns of `data`" =  
    length(covariates) > 0L && all(covariates %in% names(data))  
)  
missing_ind <- setdiff(indicators, names(data))  
if (length(missing_ind)) {  
  stop2("The following indicators are not columns of `data`: ",
```

```
    paste(missing_ind, collapse = ", ")
}
```

(`stop2()` is the package's own error helper, `R/zz_utils.R:114`.)

7 What the port got right

Worth recording, since these are deliberate improvements over the template rather than accidents:

- **Indicators from the model, not the data.** `parseModel(model)$indicators` (R/`postestimate_doTrees.R:33`) replaces the template’s `setdiff(names(data), covariates)`. The template silently treated any stray column as an indicator; the port does not, and now tolerates extra columns in `data`. The two orderings differ (`x31, x32, ..., x11` vs `x11, x12, ...`), which is why the recorded snapshot’s model formula reads `~x31 + x32 + x33 + x11 + ...` — expected, not a bug.
- **The unified dispatch is the right cut.** Branching on `influence` rather than exposing two constructors correctly reflects that the COIN and partition families differ only in how the node statistic is produced.
- **`idx_permutation()` and `ndt_dists()` were ported carefully.** The $R \times n$ orientation and the (`Sc_pool`, `Si_pool`, `mga_fit`) argument order both match their call sites, and `ndt_dists()` uses unprefixed internal calls — the convention the older `ndt_pools()` still violates.
- **B2 was fixed properly rather than patched.** Making `splitter` a `match.arg()` string closed the `logical(0)` hole structurally instead of special-casing `NULL`, and kept the one `stopifnot()` that still carries meaning. Both guard paths now produce errors a user can act on.
- **`influence_ssr` → `influence_vec`** reads better next to `influence_mat`. The stale “COIN_ssr” references in the comments (R/`helper_doTrees.R:113, :514`) should follow the rename, as should the now-incorrect provenance comment at `:535`.
- The scan cache, minbucket filtering, and `.Machine$double.eps` flooring all survived the port intact and behave correctly under test.

8 Suggested order of work

#	Why in this position
1	One line, silently wrong today, no dependencies <pre>— restore cc\$bonferroni <- isTRUE(control\$bonferroni)</pre>
2	Without it you cannot iterate on the tests at all <pre>— split ctl_mixed / ctl_part test controls</pre>
3	The signature has settled, so snapshots are now safe to regenerate; T2 fails outright once T3 lands <pre>— regenerate snapshots, drop length()==5, split expect_no_error()</pre>

#	Why in this position
4	§5.3 — kernel, argmax_split, and splitfun- wiring tests
5	§5.4 — add n_fail_split, then the negative- control test
6	§5.5 — scan- cache tests
7	§5.6 — leaf refits, then fix coef() / plot()
8	P6 — prune exports, drop :::, fix \value, validate inputs

#	Why in this position
9	Optional; only if the mixed pairs are a real study arm — numeric- cutpoint fixture